

Guia de Caracteres Coringas em Expressões Regulares (Regex)

Metacaracteres, quantificadores, âncoras, classes, grupos e exemplos práticos

1. Introdução

Uma expressão regular (regex) é um padrão de texto usado para localizar, validar, extrair ou substituir trechos de strings. Além dos caracteres comuns, que casam com eles mesmos, a regex possui metacaracteres (os chamados "caracteres coringas"), que têm significado especial e permitem descrever padrões flexíveis.

Este documento reúne os principais metacaracteres, explica o que cada um faz e traz exemplos. A sintaxe descrita segue o padrão PCRE/ECMAScript, usado em JavaScript, Java, Python, PHP, MongoDB (operador \$regex) e na maioria das linguagens. Pequenas variações podem existir entre ferramentas.

2. Resumo dos metacaracteres

Os caracteres abaixo têm significado especial fora de colchetes. Para usá-los literalmente, é preciso escapá-los com barra invertida (\).

Caractere	Função
<code>. ^ \$ * + ? { } [] \ ()</code>	Os 12 metacaracteres principais. Cada um é detalhado nas seções seguintes.

3. O ponto (.) – qualquer caractere

Caractere	Significado	Exemplo
<code>.</code>	Casa com qualquer caractere único, exceto quebra de linha (a menos que a flag <code>s</code> / DOTALL esteja ativa).	<code>c.sa</code> casa com "casa", "cesa", "c9sa"
<code>\.</code>	Ponto literal (escapado).	<code>www\.site</code> casa com "www.site"

4. Quantificadores

Quantificadores indicam quantas vezes o elemento imediatamente anterior pode se repetir.

Caractere	Significado	Exemplo
<code>*</code>	Zero ou mais ocorrências.	<code>ab*c</code> casa com "ac", "abc", "abbbc"
<code>+</code>	Uma ou mais ocorrências.	<code>ab+c</code> casa com "abc", "abbc", mas não "ac"
<code>?</code>	Zero ou uma ocorrência (opcional).	<code>colou?r</code> casa com "color" e "colour"

Caractere	Significado	Exemplo
{n}	Exatamente n ocorrências.	\d{4} casa com "2026"
{n,}	No mínimo n ocorrências.	\d{2,} casa com "12", "123456"
{n,m}	Entre n e m ocorrências.	\d{2,4} casa com "12", "123", "1234"

4.1 Quantificadores gananciosos x preguiçosos

Por padrão, os quantificadores são gananciosos (greedy): consomem o máximo possível de texto. Acrescentando ? logo depois, eles passam a ser preguiçosos (lazy): consomem o mínimo possível.

Guloso	Preguiçoso	Exemplo
*	*?	<.*?> em "a" casa com ""; <.*?> casaria com a string toda
+	+?	a.+?b casa com o menor trecho entre a e b
?	??	Prefere não casar o item opcional
{n,m}	{n,m}?	\d{2,4}? casa com apenas 2 dígitos, se possível

Existe ainda a forma possessiva (*+, ++, ?+), disponível em PCRE e Java, que não permite retroceder (backtracking).

5. Âncoras e limites

Âncoras não consomem caracteres; apenas indicam uma posição no texto.

Caractere	Significado	Exemplo
^	Início da string (ou da linha, com a flag m).	^Rio casa com textos que começam com "Rio"
\$	Fim da string (ou da linha, com a flag m).	paulo\$ casa com textos que terminam em "paulo"
\b	Limite de palavra (fronteira entre \w e \W).	\bgato\b casa com "gato", mas não com "gatos"
\B	Não é limite de palavra.	\Bgato casa com "nogato", mas não com "gato" isolado
\A	Início absoluto do texto (PCRE, Java, Python).	\Aabc
\z / \Z	Fim absoluto do texto (PCRE, Java); \Z aceita quebra de linha final.	abc\z

Dica: combine ^ e \$ para exigir que o padrão case com a string inteira. Exemplo: ^\d{5}-\d{3}\$ valida um CEP.

6. Classes de caracteres (colchetes)

Os colchetes definem um conjunto de caracteres, dos quais exatamente um deve casar. Dentro deles, a maioria dos metacaracteres perde o significado especial.

Caractere	Significado	Exemplo
[abc]	Qualquer um dos caracteres listados.	[aeiou] casa com uma vogal minúscula
[^abc]	Qualquer caractere, exceto os listados (negação).	[^0-9] casa com qualquer não dígito
[a-z]	Intervalo de caracteres.	[A-Za-z] casa com uma letra sem acento
[0-9]	Intervalo numérico.	[0-9]{3} casa com três dígitos
[a-zA-Z0-9_]	Combinação de intervalos.	Equivale a \w
[à-ú]	Intervalo de letras acentuadas (depende da codificação).	[A-Za-zÀ-ÿ] casa com letras com acento

Regras úteis dentro de colchetes:

- ^ só nega se for o primeiro caractere do conjunto.
- - é literal se estiver no início, no fim ou escapado: [a-z-] ou [a\-z].
-] deve ser escapado (\]) ou colocado logo após o colchete de abertura.
- ., *, +, ?, \$, (e) são literais dentro de colchetes.

7. Classes abreviadas (atalhos)

Caractere	Significado	Exemplo
\d	Dígito: equivale a [0-9].	\d{3}\.\d{3} casa com "123.456"
\D	Não dígito: equivale a [^0-9].	\D+ casa com "abc"
\w	Caractere de palavra: [A-Za-z0-9_] (em alguns motores inclui acentos, com flag Unicode).	\w+ casa com "nome_1"
\W	Não é caractere de palavra.	\W casa com espaço, "-", "@"
\s	Espaço em branco: espaço, tab, \n, \r, \f, \v.	a\s b casa com "a b"
\S	Qualquer caractere que não seja espaço em branco.	\S+ casa com uma palavra sem espaços

7.1 Caracteres de controle e especiais

Caractere	Significado	Exemplo
\n	Quebra de linha (line feed).	linha1\nlinha2
\r	Retorno de carro.	\r\n (quebra de linha no Windows)
\t	Tabulação.	nome\tidade

Caractere	Significado	Exemplo
\f / \v	Avanço de página / tabulação vertical.	–
\0	Caractere nulo.	–
\xHH	Caractere pelo código hexadecimal.	\x41 casa com "A"
\uHHHH	Caractere Unicode pelo código.	\u00E9 casa com "é"
\p{L}	Qualquer letra Unicode (requer suporte a Unicode).	\p{L}+ casa com "São"

8. Alternância (|)

A barra vertical funciona como um "ou": casa com a expressão à esquerda ou com a da direita.

Caractere	Significado	Exemplo
	Alternância entre padrões.	Rio São casa com "Rio" ou "São"
(a b)	Alternância delimitada por grupo.	(Rio de Janeiro São Paulo) casa com uma das duas cidades

Atenção: a alternância tem a menor precedência. `^Rio|São$` significa "começa com Rio" ou "termina com São". Para ancorar ambos, use `^(Rio|São)$`.

9. Grupos e capturas

Caractere	Significado	Exemplo
()	Grupo de captura: agrupa e guarda o trecho casado.	(\d{2})/(\d{2}) captura dia e mês
(?:)	Grupo sem captura: apenas agrupa.	(?:ab)+ casa com "abab"
(?<nome>)	Grupo nomeado.	(?<ano>\d{4}) permite acessar "ano"
\1, \2...	Referência ao grupo capturado (backreference).	(\w)\1 casa com letras repetidas, como "ss" ou "rr"
\k<nome>	Referência a grupo nomeado.	(?<c>\w)\k<c>

10. Lookahead e lookbehind

São asserções que verificam o que vem antes ou depois de uma posição sem consumir caracteres e sem incluí-los no resultado.

Caractere	Significado	Exemplo
(?=)	Lookahead positivo: deve ser seguido por.	\d+(?=%) casa com "50" em "50%"
(?!)	Lookahead negativo: não pode ser seguido por.	\d+(?!%) casa com números sem "%"

Caractere	Significado	Exemplo
(?<=)	Lookbehind positivo: deve ser precedido por.	(?<=R\\$)\d+ casa com "100" em "R\$100"
(?<!)	Lookbehind negativo: não pode ser precedido por.	(?<!\R\\$)\b\d+ ignora valores com "R\$"

Observação: o lookbehind não é suportado por todos os motores (por exemplo, versões antigas do JavaScript) e alguns exigem largura fixa.

11. Classes POSIX

Usadas dentro de colchetes em ferramentas como grep, sed, awk e PostgreSQL.

Caractere	Significado	Exemplo
[:digit:]	Dígitos.	[[:digit:]]+
[:alpha:]	Letras.	[[:alpha:]]+
[:alnum:]	Letras e dígitos.	[[:alnum:]]+
[:upper:]	Letras maiúsculas.	[[:upper:]]
[:lower:]	Letras minúsculas.	[[:lower:]]
[:space:]	Espaços em branco.	[[:space:]]*
[:punct:]	Pontuação.	[[:punct:]]

12. Flags (modificadores)

Flag	Significado	Exemplo
i	Ignora diferença entre maiúsculas e minúsculas.	/rio/i casa com "Rio" e "RIO"
g	Global: encontra todas as ocorrências, não só a primeira.	/\d+/g
m	Multilinha: ^ e \$ casam em cada linha.	/^abc\$/m
s	DOTALL: o ponto também casa com quebra de linha.	/a.b/s
x	Extended: ignora espaços e permite comentários no padrão.	/\d+ # número/x
u	Unicode: ativa suporte completo a Unicode.	/\p{L}+/u

13. Escapando metacaracteres

Para casar com um metacaractere literalmente, anteponha uma barra invertida. Em strings de linguagens como Java, a barra precisa ser dobrada, pois a própria string a usa como escape.

Literal desejado	Regex	Em string Java
Ponto	\.	"\."
Interrogação	\?	"\"?"
Cifrão	\\$	"\"\$"
Parênteses	\(\)	"\"(\)"
Barra invertida	\\	"\\""

14. Exemplos práticos

Objetivo	Expressão	Observação
CEP	^\d{5}-?\d{3}\$	Hífen opcional
CPF formatado	^\d{3}\.\d{3}\.\d{3}-\d{2}\$	Valida apenas o formato, não os dígitos verificadores
Telefone BR	^(?\d{2})?\s?9?\d{4}-?\d{4}\$	Aceita (11) 91234-5678
E-mail simples	^[w.+-]+@[w-]+\.[w.-]+\$	Versão simplificada
Data dd/mm/aaaa	^\d{2}/\d{2}/\d{4}\$	Não valida dias/meses válidos
Placa Mercosul	^[A-Z]{3}\d[A-Z]\d{2}\$	Ex.: BRA2E19
Cor hexadecimal	^#(?:[0-9a-fA-F]{3}){1,2}\$	Ex.: #FFF ou #1F3864
Espaços extras	\s{2,}	Útil para normalizar textos
URL	^https?:\/\/[^\s/\$. ?#].[^\s]*\$	Versão simplificada

15. Regex no MongoDB

No MongoDB, as regex seguem a sintaxe PCRE e podem ser usadas com o operador `$regex` ou com a notação literal `/padrão/flags`.

```
// Cidades que começam com "Rio" ou "São", ignorando maiúsculas
db.usuario.find( { "endereco.cidade": { $regex: /^(Rio|São)/i } })

// Forma equivalente com $regex e $options
db.usuario.find( { "endereco.cidade": { $regex: "^(Rio|São)", $options: "i" } })

// Combinando com $in
db.usuario.find( { "endereco.cidade": { $in: [/^Rio de Janeiro$/i, /^São Paulo$/i] } })
```

Dica de desempenho: expressões ancoradas no início (por exemplo, `/^Rio/`) podem aproveitar índices; expressões sem âncora ou com a flag `i` tendem a percorrer toda a coleção.

16. Boas práticas

- Prefira padrões ancorados (^ e \$) quando quiser validar a string inteira.
- Use grupos sem captura (?:...) quando não precisar do valor capturado.

- Evite .* em excesso; prefira classes específicas, como [^"]* ou \d+.
- Cuidado com o backtracking catastrófico em padrões como (a+)+, que podem travar a aplicação.
- Escape sempre a entrada do usuário antes de montar uma regex dinamicamente.
- Teste os padrões em ferramentas como regex101.com antes de usar em produção.
- Para formatos complexos (e-mail, CPF, HTML), regex serve para a validação básica; use bibliotecas específicas para a validação completa.

17. Tabela de consulta rápida

Símbolo	Descrição
.	Qualquer caractere (exceto quebra de linha)
^ \$	Início e fim
* + ?	0 ou mais, 1 ou mais, 0 ou 1
{n,m}	Entre n e m repetições
[] [^]	Conjunto e conjunto negado
() (?:)	Grupo com captura e sem captura
	Ou
\	Escape
\d \w \s	Dígito, caractere de palavra, espaço
\D \W \S	Negações das anteriores
\b \B	Limite e não limite de palavra
(?=) (?!) (?<=) (?<!)	Lookahead e lookbehind